Module 3 Introduction to OOPS Programming

Introduction to C++

1. *What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?*

→

Aspect	Procedural Programming	Object-Oriented Programming
Approach	Focuses on functions/procedures	Focuses on objects & classes
Data Handling	Data is mostly separate from functions	Data and methods are encapsulated together
Reusability	Limited (via functions)	High (inheritance, polymorphism)
Security	Less secure	More secure due to encapsulation
Examples	C, Pascal	C++, Java, Python

2. *List and explain the main advantages of OOP over POP.*

→ Advantages of Object-Oriented Programming (OOP) over Procedural-Oriented Programming (POP) are as follows:

- **Encapsulation** – Combines data and functions into a single unit (class), improving data security.
- **Data Hiding** – Internal details are hidden using access specifiers, reducing misuse of data.
- **Reusability** – Code can be reused through inheritance, saving time and effort.
- **Modularity** – Programs are divided into independent objects, making them easier to manage.
- **Flexibility** – Polymorphism allows the same function to behave differently in different situations.
- **Easy Maintenance** – Changes in one part of the program do not affect the entire system.
- **Scalability** – Suitable for large and complex software projects.

3. *Explain the steps involved in setting up a C++ development environment.*

→ Steps to Set up a C++ Development Environment are:

i. **Install a C++ Compiler**
   - Windows: Install MinGW or MSVC (Visual Studio)
   - Linux: Install GCC (g++)
   - macOS: Install Xcode Command Line Tools

ii. **Install an IDE or Code Editor**
   - Examples: Visual Studio, Code::Blocks, Dev-C++, VS Code

iii. **Configure the Compiler Path**
   - Set the compiler path in the system Environment Variables so the compiler runs from the command line.

iv. **Create a C++ Program**
   - Write a simple program using a .cpp file (e.g., main.cpp).

v. **Compile the Program**
   - Use the compiler (e.g., g++ main.cpp -o main).

vi. **Run the Program**
   - Execute the compiled file to check the output.

4. *What are the main input/output operations in C++? Provide examples.*
→ C++ uses the **iostream** library for input and output operations.

1. Output Operation (cout) - Used to display output on the screen.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

2. Input Operation (cin) - Used to take input from the user.

```
#include <iostream>
using namespace std;

int main() {
    int a;
    cout << "Enter a number: ";
    cin >> a;
    cout << "You entered: " << a;
    return 0;
}
```

Variables, Data Types, and Operators

1. What are the different data types available in C++? Explain with examples.

→ **a) Basic (Primitive) Data Types**

- int – stores integers = E.g. int a = 10;
- float – stores decimal numbers = E.g. float b = 5.5;
- double – stores large decimal values = E.g. double c = 12.345;
- char – stores a single character = E.g. char ch = 'A';
- bool – stores true/false = E.g. bool flag = true;

b) Derived Data Types

- Array = E.g. int arr[3] = {1, 2, 3};
- Pointer = E.g. int *p;
- Function

c) User-defined Data Types

- struct, union, enum, class

  E.g. struct Student {
  int id;
  };

2. Explain the difference between implicit and explicit type conversion in C++.
→ **Implicit Type Conversion**
- Automatically performed by the compiler.
- Converts smaller data type to larger data type.
- No data loss generally occurs.

Example:
int a = 10;
double b = a; // implicit conversion

Explicit Type Conversion
- Performed manually by the programmer using type casting.
- Converts larger data type to smaller data type.
- Data loss may occur.

Example:
double x = 5.5;
int y = (int)x; // explicit conversion

3. What are the different types of operators in C++? Provide examples.
→
1. **Arithmetic Operators** – + - * / %
   Example: c = a + b;
2. **Relational Operators** – == != > < >= <=
   Example: a > b
3. **Logical Operators** – && || !
   Example: a && b

4. **Assignment Operators** – = += -= *=
   Example: a += 5;
5. **Increment/Decrement Operators** – ++ --
   Example: a++;
6. **Bitwise Operators** – & | ^ << >>
   Example: a & b

4. Explain the purpose and use of constants and literals in C++.
Constants

- Fixed values that cannot be changed during program execution.
- Declared using const keyword.

Example:
const int MAX = 100;
Literals

- Constant values assigned directly to variables.
- Examples include integer, float, character, and string literals.

Example:
int a = 10;
char ch = 'A';

Control Flow Statements

1. What are conditional statements in C++? Explain if-else and switch.
Conditional statements control the flow of execution based on conditions.
if-else Statement

- Executes code when condition is true or false.

```
if (a > b)
   cout << "A is greater";
else
   cout << "B is greater";
```

switch Statement

- Used when multiple choices are available.

```
switch(choice) {
   case 1: cout << "One"; break;
   case 2: cout << "Two"; break;
}
```

2. Difference between for, while, and do-while loops in C++.

Loop	Condition Check	Executes At Least Once
for	Before loop	No
while	Before loop	No
do-while	After loop	Yes

3. How are break and continue used in loops?
break

- Terminates the loop immediately.

```
if(i == 5)
   break;
```

continue

- Skips current iteration and moves to next.

```
if(i == 5)
   continue;
```

4. Explain nested control structures with an example.

Nested control structures mean one control statement inside another.

Example:

```
if(a > 0) {
   for(int i = 0; i < a; i++) {
      cout << i;
   }
}
```

Functions and Scope

1. What is a function in C++?

A function is a block of code that performs a specific task.

Declaration: tells compiler about function

Definition: contains function body

Calling: executes the function

2. Scope of variables in C++. Local vs Global

Local Variables

- Declared inside a function.
- Accessible only within that function.

Global Variables

- Declared outside all functions.
- Accessible throughout the program.

3. Explain recursion in C++ with an example.

Recursion is a function calling itself.

Example:

```
int fact(int n) {
   if(n == 0)
      return 1;
   return n * fact(n - 1);
}
```

4. What are function prototypes? Why are they used?

Function prototypes declare a function before its use.

Purpose:

- Helps compiler check correct function usage.
- Avoids errors.

Example:

```
int add(int, int);
```

Arrays and Strings

1. What are arrays? Difference between 1D and 2D arrays.

Array stores multiple values of same data type.

- **1D Array:** Single row
  int a[5];
- **2D Array:** Rows and columns
  int a[2][3];

2. Explain string handling in C++.

Strings are sequences of characters.

- Using char arrays
- Using string class

Example:

```
string name = "C++";
```

3. Array initialization (1D & 2D).

1D Array:

```
int a[3] = {1, 2, 3};
```

2D Array:

```
int a[2][2] = {{1,2},{3,4}};
```

4. String operations and functions in C++.

Common functions:

- strlen() – length
- strcpy() – copy
- strcat() – concatenate
- strcmp() – compare

Introduction to OOP

1. Key concepts of OOP

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

2. Classes and objects with example.

Class: Blueprint

Object: Instance of class

```
class Student {
public:
   int id;
};
Student s1;
```

3. What is inheritance in C++?
Inheritance allows one class to acquire properties of another class.
class A {};
class B : public A {};

4. What is encapsulation? How is it achieved?
Encapsulation binds data and functions together.
Achieved by:

- Using classes
- Access specifiers (private, public)